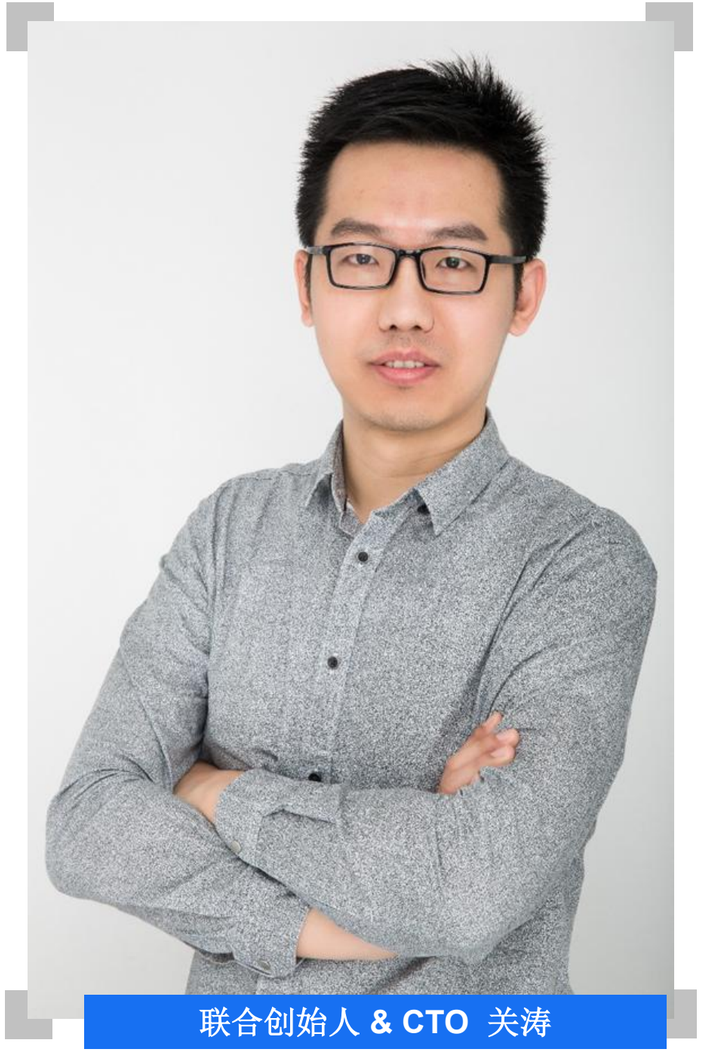
AI 时代，知识与本体的演进与思考

—— 从数据工程到知识工程

演讲嘉宾：关涛

云器科技 联合创始人 & CTO

自我介绍



联合创始人 & CTO 关涛

2006 微软

微软亚洲研究院：构建微软第一代分布式 KV 系统 **SearchRepository**（7,000 台物理服务器规模）和 Bing 搜索的分布式存储后台 **Kirin Store**（4 万台规模）

之后加入微软美国云计算和企业事业部，主持和参与开发 **Cosmos / Scope**（20 万台规模）、iScope、**Azure Data Lake**

2016 阿里云

计算平台事业部研究员，阿里巴巴通用计算平台负责人：负责阿里巴巴主线大数据平台飞天 **MaxCompute**、**DataWorks**（10 万+ 规模）

阿里和蚂蚁集团技术委员会计算平台领域组长、阿里云架构组大数据组组长；2021 年代表阿里云主持 **Forrester 云数仓评测**，首次入选全球榜单“卓越表现者象限”

2021 云器科技

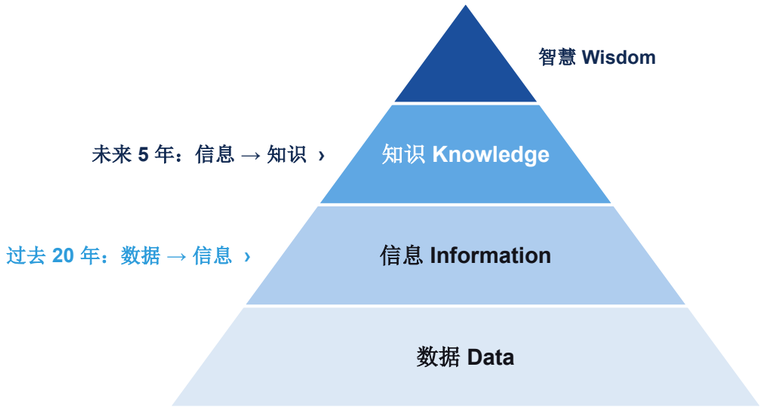
联合创立云器科技，担任 CTO —— **“基于增量的云湖仓”**数据基础设施提供商（yunqi.tech）

2026 信通院

工信部信通院**“本体与知识”标准工作组**，副组长单位

20 年分布式系统和大数据平台开发经验，著有多篇国内外会议论文和专利

2018 浙江省科技进步一等奖 · 2021 USENIX ATC Best Paper Award



DIKW · 第三次数据革命：从信息到知识

01

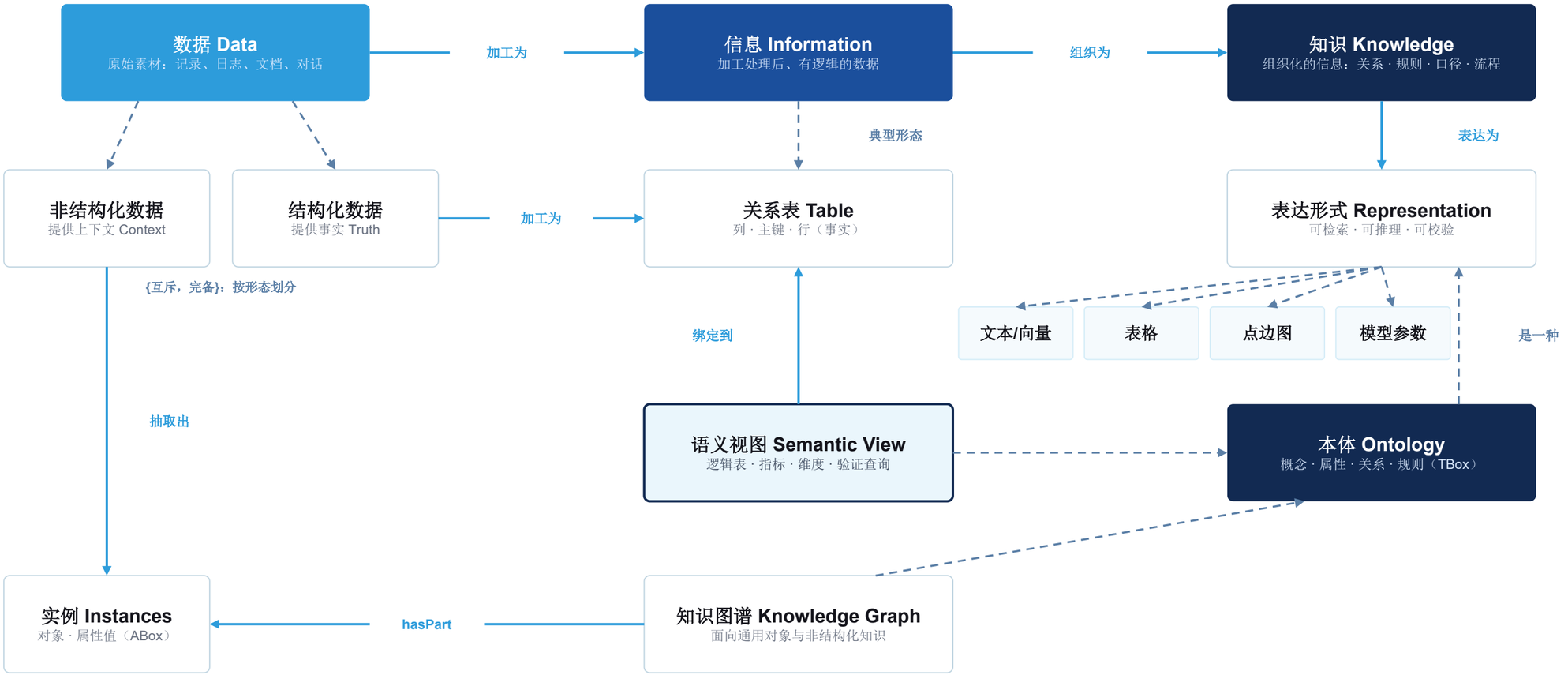
为什么是知识，知识是什么

第三次数据革命 · 数据 / 信息 / 知识 / 本体 / 语义



一张概念地图：数据、信息、知识、本体与语义

按本体论画成类图：节点是类，框内是定义或属性，虚线是子类（分类），实线是带名字的关系，右侧是公理



公理 Rules

1 **关系表** 是一种 信息
（表 = 信息）

2 **语义视图** 定义为 绑定到关系表的本体
（结构化数据之上的本体）

3 **知识图谱** 定义为 含实例的本体
（本体 + 实例）

4 **关系表 + 语义视图** 推出 知识
（表 + 语义 = 知识）

5 **数据的形态** 互斥且完备：结构化 / 非结构化

图例 框 = 类：第一行是类名，第二行是定义或数据属性；右侧 = 公理（定义 / 推出）。

虚线 = 子类（is-a）：分类关系，“关系表是一种信息”　　实线 = 对象属性：带名字的关系，如 加工为 / 绑定到 / hasPart

两种线都是关系，但层级不同：子类是概念之间的包含（每个 X 都是 Y），对象属性是实例之间的联系（X 经过某种动作或关联得到 Y）。

知识的表达（建模）：同一份知识，可以有多种形态

同一个事实：**Meta 2023 年总营收 1,349 亿美元**

2021年，Meta的总营收为1179亿美元。
2022年，Meta的总营收为1166亿美元。
2023年，Meta的总营收达到1349亿美元。

（约100个字节）

	Meta年营收（亿美元）
2021	1179
2022	1166
2023	1349

（约10-20个字节）

2021年，Meta的总营收为1179亿美元。
2022年，Meta的总营收为1166亿美元。
2023年，Meta的总营收达到1349亿美元。

（约33,000个字节 in PNG）

表达方式	二维关系表	标量 / 倒排索引	向量	点边（图）	模型参数
表达能力	中（结构化数据）	中（半结构化数据）	高（任何数据）	高（关系与路径）	高
存储方式	二维表、行存 / 列存	倒排索引	向量索引	图存储 / KV	大模型内（通过训练）
计算方式	关系代数	匹配检索	距离计算	图计算	提示词查询（Prompt）
使用难度	中，灵活	低，不灵活	低，不灵活	高，灵活	极低，非常灵活
准确度	高	高	中	高	低
信息密度	高	中	低	高	中
可解释性	高	中	低	高	低
可控性	高	高	低	高	低
建模难度	中	低	低	高	极高

好　中　差　底色按该维度的好坏方向着色（建模难度“高”是差，可解释性“高”是好），不是数值高低。

数据表达，向高准确性、高信息密度、高可解释性、高可控性流动（从数据到知识），**但没有一种形态包打天下 —— 表达方式决定可读、可检索、可推理的能力与代价**

02

知识的五类路线

当前没有统一标准 —— 五种不同的建模与运行方式



知识（本体）建模有标准吗？ 目前有很多标准，但并不统一（Aka 当前业界没有标准）

准确地说：有表达、查询和约束规范，但没有产业共同接受的企业本体建模、运行与维护标准。

当前没有任何一套标准，同时规定

建什么 · 怎么运行 · 如何裁决 · 怎样持续维护

01 W3C 标准化的是表达

RDF / OWL / SHACL 规定机器如何表达、推理和校验；不规定企业应该采用哪一种业务对象与运行模型。

02 产业实现彼此不同

Palantir 建对象与 Action；Semantic View 建指标与 SQL；Genie 建知识片段并排序——并非同一标准的实现。

03 开放交换仍在形成

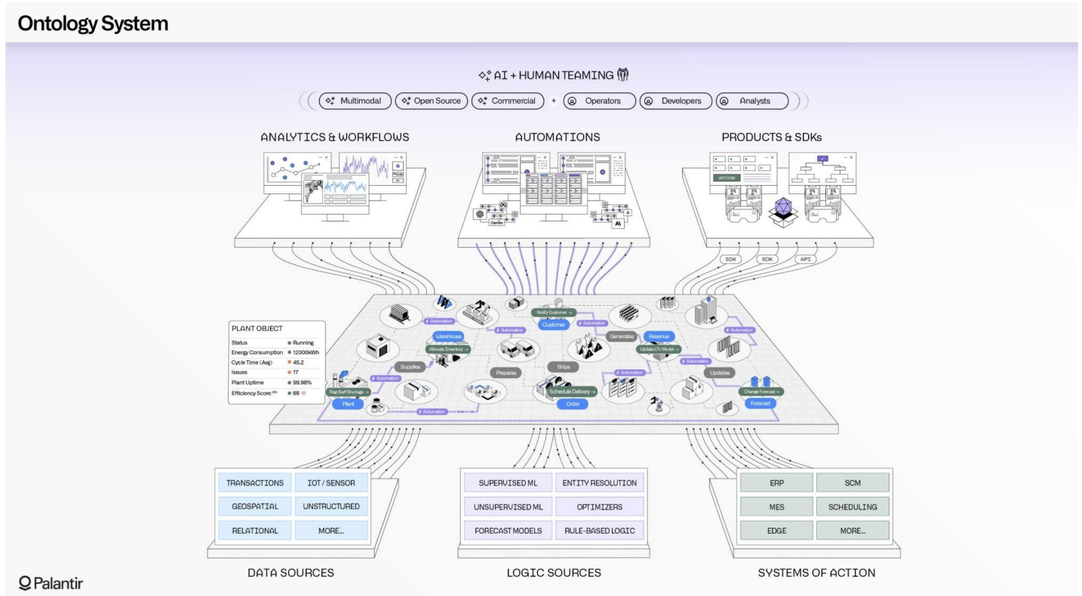
Apache Ossie 仍处 Incubating，当前重点是跨工具交换；它也不提供统一运行时与企业本体方法论。

因此，后面的五类系统不是同一标准下的五个实现，而是五种不同的本体 / 语义建模模型。

Sources: W3C Semantic Web Standards · Apache Ossie (incubating) · Palantir / Snowflake / Databricks official docs

Palantir Ontology：数字孪生与行动系统

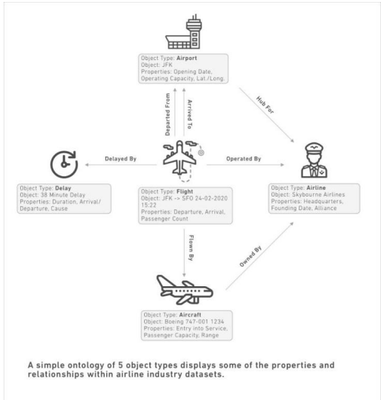
对象、属性、链接构成语义元素；行动与函数构成动力元素 —— Agent 不只查询业务，还能安全地改变业务



来源：Palantir Docs · Architecture Center · The Ontology system（www.palantir.com/docs/foundry/architecture-center/ontology-system）

读图：三条输入 —— 数据源（交易、IoT、地理、非结构化）、逻辑源（ML、优化器、规则）、行动系统（ERP、MES、SCM、调度）；

本体居中，对象之间带 Action 与 Automation；三条输出 —— 分析与工作流、自动化、产品与 SDK，形成读写闭环。



来源：Palantir Docs · Ontology core concepts（航空本体示例：5 个对象类型）

语义元素

对象类型 · 属性 · 链接类型；接口提供多态

动力元素

行动类型 · 函数 · 动态权限；Action 写回源系统

建模中心 – 属性式对象模型 · 通用建模

Object / Property / Link / Action：把所有要素都建模了 —— 包括行动，不只是数据和知识；实例物化为可操作对象

强项 –全面建模，行动包含层，本体运行引擎

引擎里包含驱动行动的部分：已经从表达层深入到**运行层**：语义、实例、权限、Action 与写回统一在一个平台里，**全面性和深度是五条路线里最好的**

边界 – 垂直领域、不开放、高成本

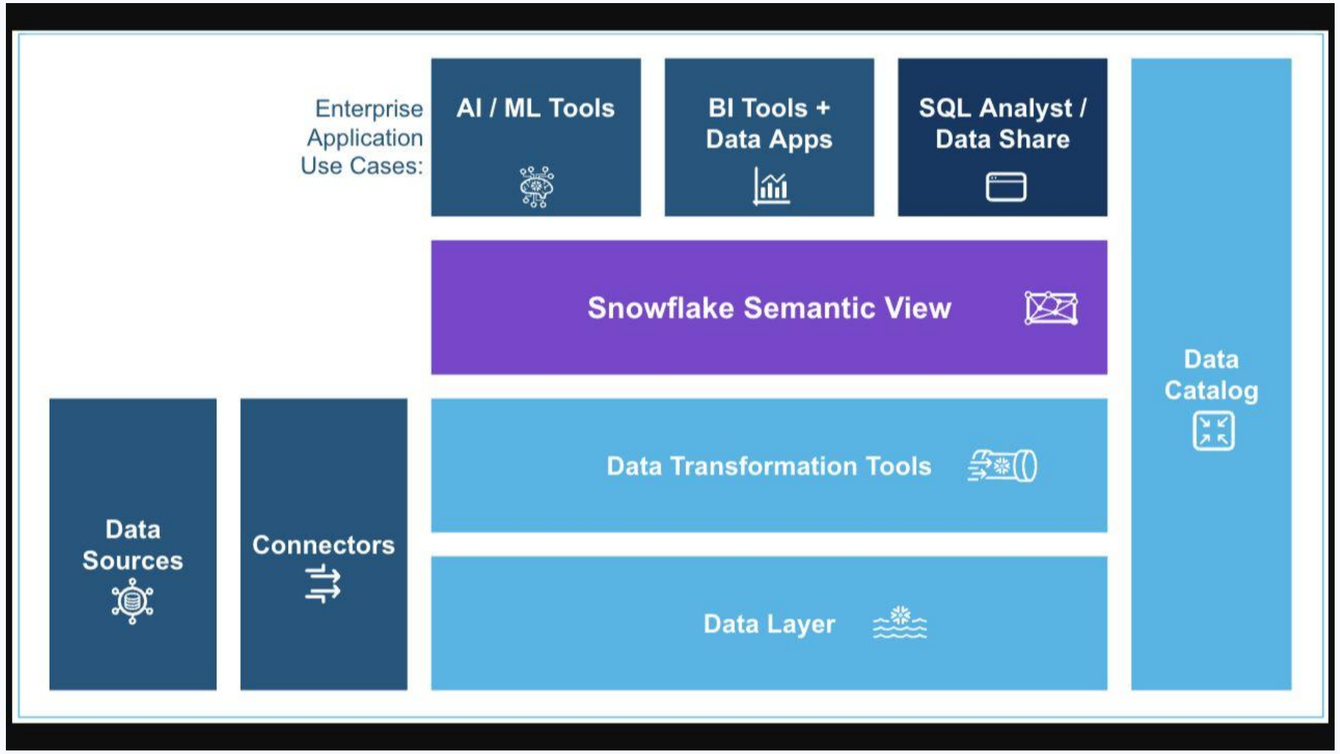
实施重、实例规模大、平台封闭；是高成本的治理闭环，不以开放语义交换为目标

路线判断

适合高治理的运营闭环；不是起点

Snowflake Semantic View：数据库中的分析语义对象

用逻辑表、关系、维度和指标描述业务语义；不物化实例，查询时编译为 SQL



建模中心 – 属性式对象模型 · 分析语义（对象 = 逻辑表）

Logical Table + Relationship + Fact / Metric / Dimension；实例仍留在源表，查询时编译为 SQL

强项 – 确定性 + 准确性

分析口径确定、结果准确；原生 RBAC 与审计，对 NL2SQL 直接有效

边界 – 仅在结构化数据领域

主要覆盖仓内结构化分析语义，不承担业务实例写回与行动编排

路线判断

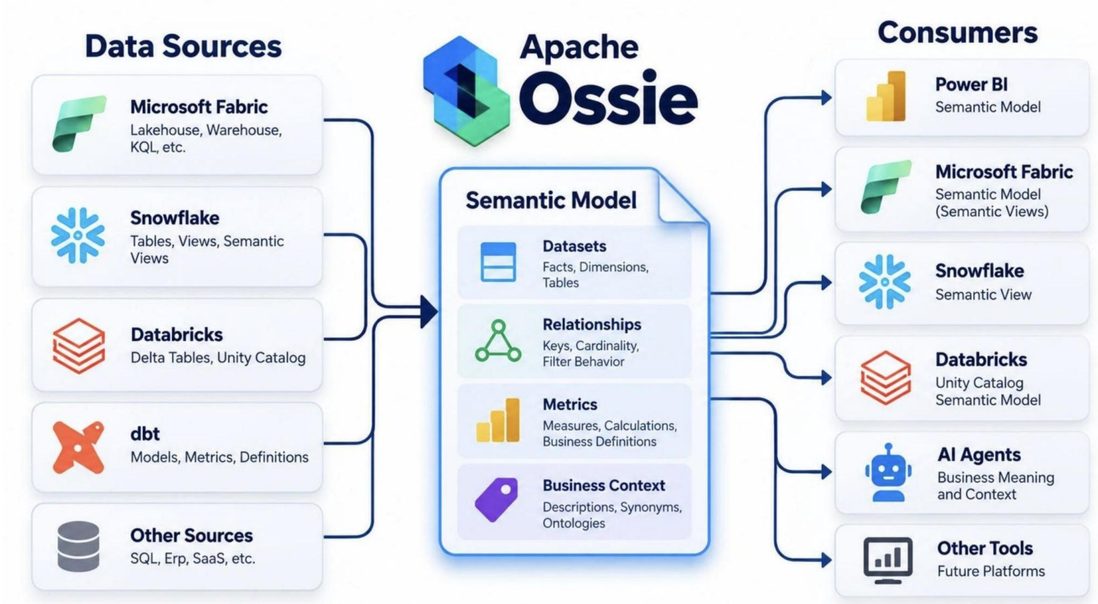
结构化分析语义的主流起点：确定、准确，改造成本适中

Yunqi Tech Data Platform

Source: Snowflake Semantic Views | https://docs.snowflake.com/en/user-guide/views-semantic/overview

Apache Ossie：跨工具交换语义模型的开放规范

厂商中立的 YAML 交换格式；最新规范把**本体层与核心语义层分开** —— 目前看得到的、从结构化语义向本体延伸的一条路径



来源：LinkedIn 社区分享的 Apache Ossie 生态示意图（数据源 → 语义模型 → 消费方，含 AI Agents）

查询接口 / MCP

两层都向 Agent 和 BI 暴露：

· 维度式接口：结构化查询对象，Agent 无法自造 join

· 关系式接口：SQL + MEASURE()，治理过的指标

· 本体：业务概念与含义，在指标之上

Agent 通过 MCP 工具调用，带类型化错误

建模中心 – 实体模型（事实式，也是知识模型的一种）

本体层：EntityType / Role / Relationship 以事实（关系陈述）建模，没有“属性”，比属性式对象模型更宽泛；核心语义层仍是属性式（与 Semantic View 同构）

强项 – 确定性、准确性 + 开放标准

同样是 Schema / SQL 编译，口径确定、结果准确；厂商中立的**开放标准**，一次编写、各处查询，不绑定任何引擎

边界 – 结构化范畴，最近增加Ontology层

和 Semantic View 一样，目前局限在结构化数据；本体层近期才刚起步；不是查询引擎，也不是语义运行时

路线判断

企业的另一个**起点**：开放、无锁定；

本体层 Ontology 偏业务定义

概念 EntityType / ValueType · 关系 + 角色 · 规则（requires / derived_by / identify_by / extends）

映射 Mapping 两层之间的桥

ontology_ref + semantic_model_ref；Object mapping → 字段 SQL 表达式，Link mapping → 连接树

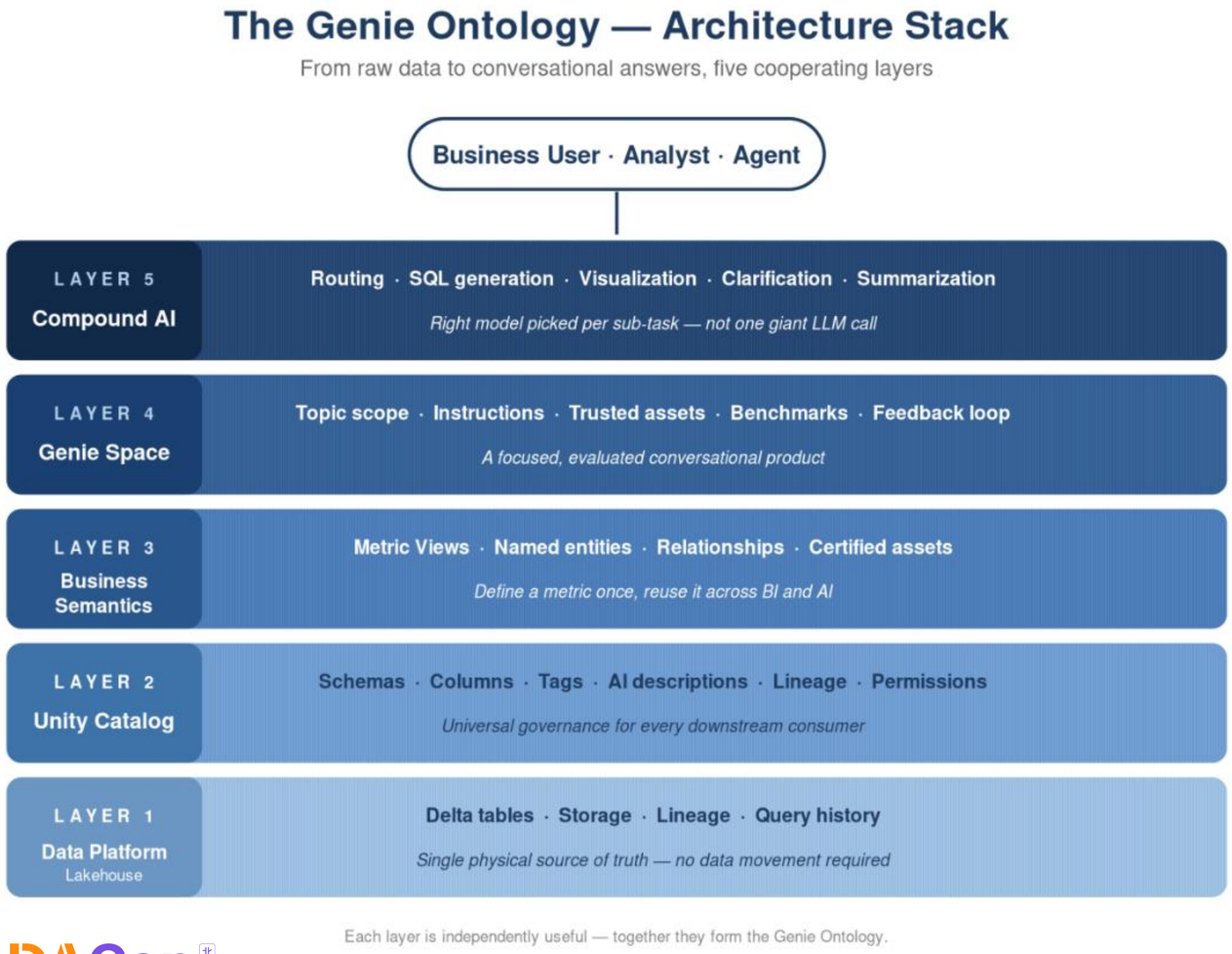
核心语义层 Semantic 偏数据定义

Dataset / Field · Relationship · Metric / Dimension；YAML / JSON，实例留在源表

规范来源：Apache Ossie (incubating) · ossie.apache.org；Ontology Specification（github.com/apache/ossie · ontology/ontology.md）与 Core Semantic Model Spec；查询接口层次依据社区工作组提案

Databricks Genie Ontology：从使用痕迹中推断上下文

从查询、Dashboard、Pipeline 与协作系统中抽取知识片段，并按权威性与时效排序



建模中心 – 混合模型

既包含一部分结构化 Semantic（Metric View），也包含企业知识（Knowledge Snippet + 资产 / 用量关系）—— 比只做 Semantic 更全面；不声明统一概念模型

强项 – 覆盖面广 + 主打自动化生成

覆盖面更广；大量自动化生成能力，能从真实使用持续学习与更新，建模成本低

边界 – 相对全面

排序不等于验证，难以表达非可加、基数等需要强契约的约束

路线判断

比较**普适的模式**：适合补齐长尾上下文；认证真值仍需语义内核

Agent Memory：自由文本中的个体知识与经验

无需预定义结构；适合个体上下文，但不能承担组织级唯一真值

Memory

You

Preferences	Tony's stated preferences for ho…	Updated Jun 30
Profile	Who Tony is — role, company, pro…	Updated Jul 10

Topics

Interests	Tony's hobbies and personal inte…	Updated Jul 10
Recent Work	Work activity and prospecting th…	Updated Jul 26
Tooling And Setup	Tony's remote development envir…	Updated Aug 3
Travel	Tony's travel patterns, frequent-…	Updated Jul 19

Areas

Ai Functions Lakehouse	Tony's research and customer-fa…	Updated Jul 2
Ai Observability Infra	Tony's deep research into AI/LL…	Updated Jul 8
Bayarea Worldcup Trip	Tony's June 2026 Bay Area trip —…	Updated Jun 17

← Memory

Ai Observability Infra Delete

Summary

Tony's deep research into AI/LLM observability platforms and storage architecture, including Langfuse self-hosting integration

Details

- Investigated the AI/LLM observability landscape at an architectural level, covering Langfuse, LangSmith, Braintrust, Arize Phoenix, and OpenTelemetry GenAI semantic conventions
- Focus on storage engine design constraints for evaluation-as-first-class-citizen workloads
- Noted Langfuse's ClickHouse acquisition as architecturally significant
- Integrated Claude Code's OTEL support with a self-hosted Langfuse instance
- Worked through gRPC/HTTP protocol mismatches and Basic auth configuration during the integration
- Researched memory system evaluation using LongMemEval, covering retrieval metrics (R@5, NDCG@5), LLM-as-judge scoring, and a three-layer evaluation framework
- Interest in building or evaluating agent memory systems
- Researched the OLAP-for-AI-infra competitive landscape including Apache Doris's AI observability strategy, SelectDB's Langfuse fork, StarRocks, GreptimeDB, and MyScale
- Investigated OpenTelemetry GenAI Semantic Conventions for DAU reporting and trace/eval storage design
- Noted ClickHouse's acquisition of HyperDX in addition to Langfuse
- Noted Langfuse v4 ClickHouse-native architecture post-acquisition
- Studied Braintrust's eval-first CI/CD model
- Noted OTel GenAI Semantic Conventions use the gen_ai.* namespace and treat evaluation as a first-

I work Pacific time hours

主流方案的对比：设计目标不同，形态就不同

本体没有统一标准的结果：五类系统各自选择了不同的目标、保证与自由

Palantir Ontology

属性式对象模型，**通用建模**：连行动一起建

行动能力：从表达层深入到运行层，全面性与深度最好

垂直领域、高成本的治理闭环，不是开放的起点

Semantic View 家族

属性式对象模型，对象 = 逻辑表（分析语义）

强项：**确定性 + 准确性**，查询时编译 SQL

只针对结构化语义：**主流起点**

Apache Ossie

本体层 **事实式实体模型**，更宽泛；语义层同 SV

同样确定、准确，且是**开放标准**，无锁定

仍限结构化，本体层刚起步，无运行时；**另一个起点**

Databricks Genie

混合模型：结构化语义 + 企业知识片段

覆盖面广，大量自动化生成，持续学习

推断不等于认证；比较**普适的模式**

Agent Memory

情景 / 语义记忆（episodic / semantic）

自由文本、个体偏好与经验

无 Schema，不承担组织级真值

强保证
口径唯一 · 可验证 · 可写回

→

开放接口
跨工具交换 · 不绑定引擎

→

高灵活
覆盖广 · 低建模 · 弱保证

每向右一步：用一种保证，换一种自由

核心总结：不同方案的设计目标和方向不同，所以在不同场景上会采用不同的知识建模形态。
既然知识有多种建模形态，企业就需要一个通用的框架来建设知识 —— 这是第四章要回答的问题。

五类系统真正不同的是知识表示与运行方式

“图”只是载体：先看有没有显式 TBox / ABox / Action，再看语义来自人工声明还是机器推断

模型维度	Palantir	Semantic View	Apache Ossie	Genie Ontology	Agent Memory
知识表示形态	**Schema based** 对象本体 TBox + ABox + Action 一体	**Schema based** 分析语义模型 TBox-like；ABox 留在源表	**Schema based** 交换契约 语义层 + 事实式本体层	**Semi-Schema** 推断上下文 片段 + 关系 + 排序	**Schema-Less** 自由文本记忆 无显式 TBox / ABox
概念建模范式	属性式对象模型	属性式对象模型（对象 = 逻辑表）	本体层：事实式实体模型；语义层：属性式（同 SV）	混合：属性式分析对象 + 无 Schema 知识片段	无显式概念模型
关键原语	Object / Property / Link / Action	Table / Relationship / Fact / Metric / Dimension	Dataset / Field / Metric + EntityType / Role	Snippet / Asset / Usage Signal / Rank	Markdown / Text Chunk
实例与运行	实例物化并索引；运行引擎驱动 Action 写回	数据留源表；查询时编译 SQL	YAML / JSON；无运行时；导入方执行	片段库 + 排序服务；回答时裁决	目录树 / 检索 + LLM 理解
语义生产	人工建模	人工声明 + Autopilot 提案	人工声明	机器自动生成 + 人工增权	LLM 自写自读
正确性 / 裁决	Schema + 权限 + Function / Action	Schema + Compiler + RBAC：确定、准确	Schema 校验：确定、准确；执行由导入方承接	排序不等于验证	无结构校验与口径裁决
最适场景 / 硬边界	高治理运营闭环 垂直、高成本；不是开放起点	主流起点：BI / NL2SQL 限结构化语义；无 Action	开放标准：另一个起点 限结构化；无运行时	普适：长尾 + 部分结构化 推断不等于认证	个体记忆 不能承担组织真值

Yunqi Tech Data Platform

Source: Singdata Research | 《语义层与本体调研白皮书》3.1–4.3；《本体与语义模型对比》2–4

03

知识建设的挑战与应对

两个挑战 · 两个案例 · 四个架构共识



挑战一：知识的获取 —— 最难的知识不在文档里，在人脑子里

知识的获取：尽量不要求人写文档，只要求人正常工作 —— 让“dump”知识的代价趋近于零

知识的三层供给：越往下量越大，也越难拿到

容易	**已经写下来的** 文档、Wiki、制度、SOP	蒸馏不难 —— 案例二
中等	**在系统里的** 查询、Dashboard、代码、配置	从使用痕迹归纳 —— Genie 路线、验证查询（案例一）
最难	**在人脑子里的** 口径、经验、判断、习惯性流程	量最大、最难；人自己也没意识到该写出来

解法：Trace —— 人和 AI 的每一次交互，都是知识的原料

提问、纠正、确认、选择，甚至日常沟通，都被 trace 下来；从 trace 里蒸馏口径、偏好、流程，验证后入库。
第二次数据革命的原料是 Log（人的行为），第三次数据革命的原料是 Trace（人的认知）。

一段真实的 Trace，和从中蒸出的知识

用户 本月 GMV 是多少？

Agent 本月 GMV 为 1,200 万。

用户 不对，**GMV 不应该包含退款订单。**

系统识别：指标口径纠偏
候选规则「GMV 扣减退款」→ 管理员确认 → 评测 → 语义版本 +1

Trace 里常见的五类线索

指标口径纠偏	“GMV 不应该包含退款”
时间口径纠偏	“按支付时间，不是创建时间”
过滤条件补充	“要排除测试商家”
同义词补充	“成交额就是 GMV”
关联关系补充	“订单表按 store_id 关联门店表”

知识不只是语义和问答
还有大量程序性知识：流程、经验、技能、工具用法。流程是人做出来的，不是写出来的 —— 它几乎只在 Trace 里才看得见。

保鲜也在同一个机制里完成：**Trace 持续发生，知识持续更新。**

案例一：从“验证查询”蒸馏语义视图 —— 自动化、可验证、零编造

让大模型阅读企业已有的「业务问题 + SQL」，自动生成数据的业务说明书 —— 3 个公开基准 · 4 轮实验 · 2 轮独立数据审计

1 为什么做这件事

语义视图是 NL2SQL 与语义层的地基，目前靠专家手写 —— 贵、慢，是冷启动的第一瓶颈。

而每家企业都天然沉淀着验证查询：业务问题携带语义，确认过的 SQL 携带结构事实。

做法：大模型只把 SQL 当文本阅读、不执行；按表分批蒸馏再机械合并；每条结论带证据编号，证据不足就留空待人工复核。

可持续迭代

语料攒多少、蒸多少；哪张表缺语料自动记账、定向补充 —— 知识随使用不断长全，而不是一次性工程。

高精确

SQL 里出现过的几乎全部蒸出；没出现的绝对不碰。不确定就不猜：订单明细表的复合主键证据不足，模型留空待人工确认，而不是编一个。

2 怎么做 —— 一个真实的例子

输入 · 验证查询（编号 q039）

“秋季学期开设了哪些科目？授课教师和邮箱是什么？”

```
SELECT ccso.SUBJECT_TITLE, e.FULL_NAME, e.EMAIL_ADDRESS …
FROM EMPLOYEE_DIRECTORY e
JOIN COURSE_CATALOG_SUBJECT_OFFERED ccso
  ON ccso.RESPONSIBLE_FACULTY_MIT_ID = e.MIT_ID …
```

↓ 大模型蒸馏 · SQL 只作文本阅读，不执行

输出 · 语义视图条目

关系 **开课记录表 →（多对一）→ 教师目录表**
连接键 RESPONSIBLE_FACULTY_MIT_ID = MIT_ID
业务含义 每条开课记录对应一位负责教师；教师可负责多门课
置信度 高（连接条件在查询中显式出现）

证据：q039 —— 每条知识可追溯到源查询，可逐条复核；与 MIT 标准答案一致

3 效果

F1 = 1.0
关系结构完整还原
表名列名全部换成陌生代号仍全对（TPC-H）；与保险专家手工建模逐条一致（44 组问答 → 13 张表）

97–100%
精确率 · 零编造
MIT 真实数仓：不给查询、只给表名的对照组才出现编造（85%）

×2.3
覆盖
分批蒸馏、按表蒸馏再合并，突破单次输出约 50 条的瓶颈；“两端一致”过滤后精确率 95.8%，达到自动入库标准

理论上限 50.8%：查询里出现过的关系才可能被蒸出（MIT 标准答案共 197 条关系）—— 所以要和 Trace、文档互补。

数据来源：MIT 校园数据仓库（BEAVER 基准）真实用户查询、TPC-H、保险领域数据集；读数出自公开基准实测（experiments/OVERALL_REPORT.md）；产物遵循 Apache Ossie 格式。

挑战二：知识的加工 —— 有了 Raw Data，不等于 AI-Ready，也不等于知识

文档里抽出来的东西，还不是知识 —— 用一个例子，走一遍抽取、转换、索引、评估、服务

Raw：一份业务文档里的两段话

《交易指标口径说明 v3》

“GMV：用户下单成功的商品金额合计，按下单时间统计。”

《2025 Q3 口径勘误》

“GMV 自 Q3 起剔除退款，改按支付时间统计；报表里的‘成交额’即 GMV。”

另附一段 SQL 示例：SUM(dgmv) … WHERE is_test_seller = 0

大模型抽出来的：三条候选

① GMV = SUM(金额)，按下单时间

② GMV = SUM(金额) − 退款，按支付时间

③ 别名：成交额 = GMV

互相冲突；没有绑定到表和列；没有证据编号；不知道哪条还有效 —— 这还不是知识

要成为知识，还差四步

绑定：GMV 对应哪张表、哪一列

裁决：两个版本哪个有效、从何时起

校验：金标题库、证据校验，错了能发现

发布：带版本、可追溯，Agent 才敢用

同一个例子，走五步 —— 知识工程本质上是非结构化到结构化的过程，所以结构化数据的管线、分层、增量、评估都能复用

抽取

读两段话和 SQL 示例 → 三条候选（指标、表达式、别名），每条带来源段落编号

AI / 大模型驱动

转换

归并同名对象；绑定到 dm_trade_order_1d.dgmv；冲突只标不裁；补单位（元）、别名（成交额）

AI / 大模型驱动

索引

结构化部分进语义视图（指标 / 维度 / 过滤）；原文进全文与向量索引；中文业务词 ↔ 英文标识符桥接

评估

金标题库跑一遍：口径正确率、越界拒答；证据校验器：引用的表列必须有据可查；冲突交人工裁决

AI / 大模型驱动

服务

证书 PASS → 语义视图 v+1：GMV = SUM(dgmv)，剔除退款、按支付时间，带证据编号，供 NL2SQL / Agent 调用

Raw 的文档抽取出来，并不等于有了知识 —— 绑定、裁决、校验、发布之后，才是 Agent 能可信消费的知识。后面两个案例，就是这条路走到底的样子。

案例二：某头部内容平台 —— 把业务文档蒸馏成语义视图，而不是直接检索

25 篇业务文档 · 32 道正向题 + 14 道负向题 · 文档检索 vs 结构化语义视图，同尺对照

1 为什么做这件事

文档已经整理得很好，RAG 召回的效果却不好。

不是知识不够，是**知识太多**：同一个指标三四种相似描述散落在 25 篇文档里，召回一次带出一堆相似信息。

同一个 Agent 既要写 SQL，又要在相似指标里甄别，还要做归因分析 —— **注意力被分散**，在相似信息干扰下判断出错。

做法：把整理好的业务文档再蒸馏一层，成为结构化的 Semantic View。

进了语义视图，命中即唯一

指标、维度一旦蒸进 SV，命中就是独一无二的，Agent 不再需要在相似描述里甄别；每条带证据，越界允许拒答。

同一指标多种描述，交给专门的阶段

语料里 7 个指标各有 2 套以上合法写法。蒸馏平台用变体组归并 + 多次投票（n = 3）专门处理这件事：冲突只标不裁，交人工确认。

2 怎么做 —— 一个真实的例子

输入 · 业务文档片段 + SQL 示例（已脱敏）

“客单价（ATV）= 成交金额 ÷ 日活跃购买用户，按支付日统计，剔除测试商家。”

```sql
-- 文档里的 SQL 示例
SELECT ROUND(SUM(dgmv) / SUM(dab), 2) AS atv
FROM dm_trade_order_1d
WHERE is_test_seller = 0 AND dt = ${bizdate}
```

↓ 大模型读散文、sqlglot 解析 SQL · 确定性校验器裁决：引用的表和列必须有据可查

输出 · 语义视图条目

指标 **atv · 客单价（元） = SUM(dgmv) / SUM(dab)**
表 dm_trade_order_1d 观测列 dgmv · dab · dt · is_test_seller
常用过滤 is_test_seller = 0 别名 客单价 / ATV
越界提问 “碳排放是多少？”→ 不在 SV 中，拒答，不编造

证据：数据资产.md #para · SQL示例.md —— 每条知识可追溯到源文档，可逐条复核

3 效果

≈2× **准确率**
指标口径正确率 38.1 → 73.4；选表 F1 80.3 → 85.7（向量文档检索 → 语义视图）

−52% **Token 成本 · 每条正确口径**
24.5k → 11.7k token；每题 token −8%，其中输出 −17%

−21% **执行效率 · 长尾耗时 p90**
38.7 → 30.5 s；p75 −29%，均值 −16%

数据来源没有增加，反而变少了：同一批 25 篇文档，SV 上下文比召回的文档块还短（≈ 3.3k vs 3.9k token）。整理之后，准确率、效率、token 三项同时达标；294 次越界生成零编造。

读数来自内部评估（V2_REPORT §3.10 及生成缓存 gen_runs_v2.jsonl 按条件汇总，repeats = 3，2026-09）；实验中模型直读 SV 文本并自写 SQL，未走引擎编译路径，为保守下限。客户名称与表名按约定脱敏。

架构共识一：蒸馏过程用 Agent Loop，还是 Pipeline？

开发期让 Agent 探索，生产期跑确定性管线 —— Agents in Dev, Artifacts in Prod

	Agent Loop 开发期 · 本地	确定性 Pipeline 生产期 · 平台
谁做决策	模型自主循环：下一步做什么由模型决定	编译产物（DAG）决定；循环由确定性校验器控制
适合做什么	探索语料、标定提示词、定契约与校验规则	批量抽取、调度重跑、分片重试、审计
模型的角色	既做抽取，也做流程决策	只做抽取（AI_EXTRACT 有界算子：投票 + 修复上限）
可控性	不可预测：成本、时延、结果都随模型波动	可压测、可复现、可回滚；同一产物两套执行器

1 探索
Agent Loop 读语料、试提示词

2 编译
manifest：DAG + 契约 + 内容哈希 + 钉死的提示词

3 校验
金标评测、证据校验、人工确认

4 生产
调度执行同一产物；决策权不在模型

我们自己的实验证据

把决策权交给模型自主循环的尝试，全部负效。

有界修复环（校验失败带错误反馈重试，有硬上限）可以展开成固定 DAG —— 这是“可编译”的前提。

同一份编译产物，本地 runner（开发态）和平台工作流（生产态）执行同一个 DAG —— 和 dbt 一个道理。

行业参考

Anthropic, Building effective agents：区分 workflow（预定义代码路径）与 agent（模型自主决定过程）；能用 workflow 就不要用 agent。

dbt：compile → manifest → run，开发与生产执行同一份产物。

开发期允许 **Agent** 探索；生产期运行静态、确定性、可回滚的产物 —— 模型仍在管线里工作，但不再做流程决策。

架构共识二：知识工程/蒸馏管线 – 奖牌架构（Medallion Model）

知识工程本质上是非结构化到结构化的过程，所以整套结构化数据处理技术栈（管线、分层、增量、评估）都能复用

三层覆盖全部数据

Bronze、Silver、Gold 三层兼顾管理性与灵活性。

统一元数据

底层只有一套元数据。

AI 成为新引擎

AI 函数与分析引擎并行工作。

增量计算 – 持续更新成为可能

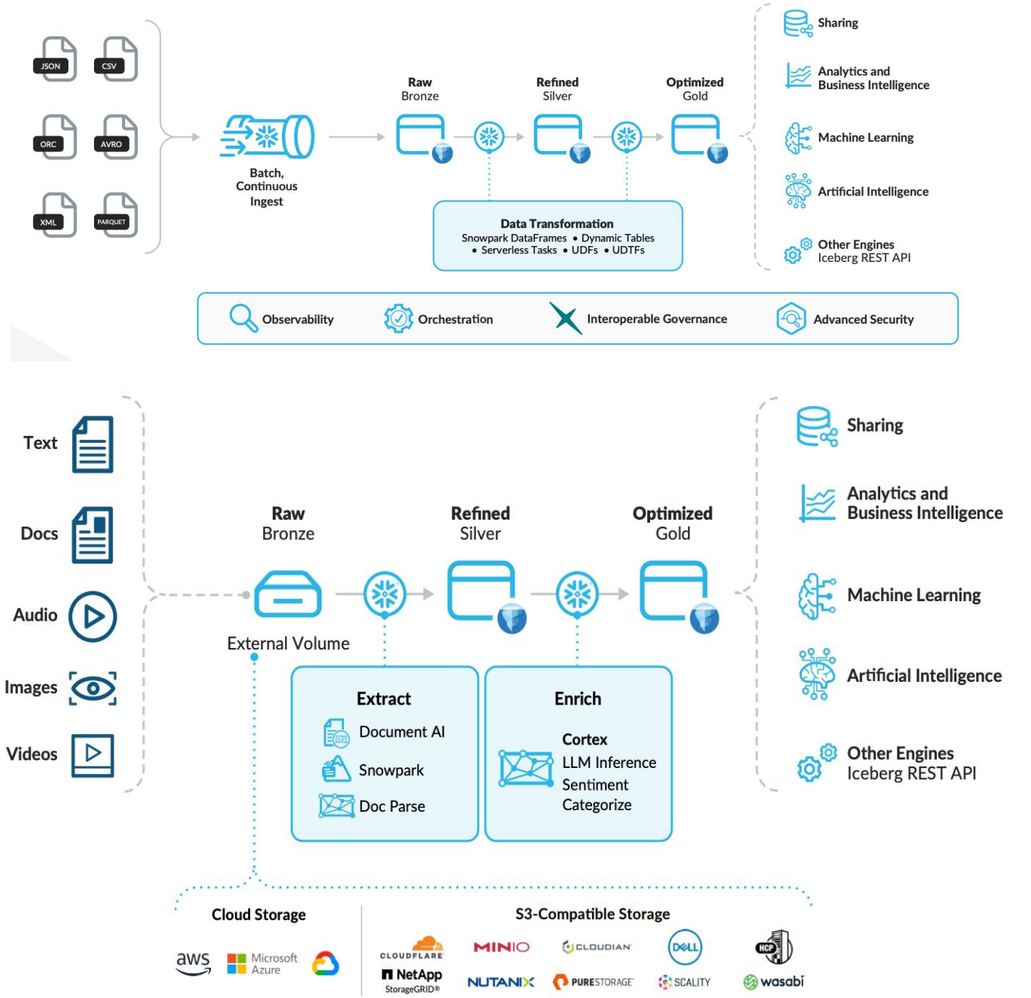
同时兼顾新鲜度与成本效率。

收益

灵活性：从原始数据存储到高级分析
数据质量：每一层逐步提升可信度
成本效率：把资源花在高价值的 Gold 层转换上
增量 ETL：分阶段处理，降低复杂度，提升扩展性

行业参考

2024年，Databricks/Snowflake 面向数据与 AI 的参考架构：同一条 Bronze / Silver / Gold 管道同时处理结构化与非结构化数据，AI 作为新引擎与分析引擎并行。



架构共识三：评估驱动 + 持续更新 —— 知识平台不是一次性交付

没有“今天建成、以后一直对”的知识平台：输出是概率性的，业务又在变 —— 评估是门禁，更新是常态

评估驱动：从测试驱动（TDD）到评估驱动（EDD）

输出不确定、没有明确标准 —— 静态测试已失效，需要持续评估

	TDD · 测试驱动	EDD · 评估驱动
核心逻辑	确定性输入 → 确定性输出	自然语言输入 → 概率性、动态输出
时机	发布前一次性静态测试，通过即完成	覆盖离线开发与线上运行的持续治理
解决的痛点	代码错误、崩溃、性能瓶颈	“口径对不对”“任务完成没有”“升级后变好还是变差”

评估的三件套（案例一、二都在用）

1. **金标题库**
   正向 + 负向双套件（32 + 14 题）；等价类 / 变体组评分，消除约定伪影
2. **证据校验器**
   每条知识引用的表和列必须有据可查；证据不足留空，不猜
3. **质量证书**
   证书 PASS 才能物化、发布；同尺对照，升级后可比

持续更新：建成只是 v1，之后每一天都在变

Trace 持续发生 → 候选 → 评估 → 版本 +1，和数据管道一样增量

发现
Trace 持续发生：新文档、新查询、一句“不对，GMV 不含退款”

候选
增量重抽：改 1 篇文档只重抽 1 篇；内容哈希钉住未变的结果

评估
语义变更回标定：金标回归 + 证据校验 + 同尺对照

发布
证书 PASS → 版本 +1；可回滚、可审计、可比较

知识平台不是一次性的软件交付：没有“建成之后就一直对”—— 评估是门禁，更新是常态，和数据管道一样增量、版本化、可回滚。

架构共识四：Eco-loop —— 知识在使用中被持续生产

一个环：使用产生 Trace，Trace 蒸馏为知识，知识回到使用



这个环的三个要点

1 入口是 Trace，不是文档

知识的供给来自真实使用，不要求人额外写东西；程序性知识只在这里看得见。

2 验证是门禁，不是排序

金标题库 + 证据校验 + 质量证书：通过才发布。排序（Genie 路线）补长尾，不裁决真值。

3 增量，而非全量重跑

改一篇文档只重抽一篇；语义变更回标定、重编译、版本 +1 —— 和数据管道一样。

第五章：三个产品模块，就是这个环的产品映射 —— SingSight = Trace + 评估，AI-ETL = 蒸馏，知识 = 入库 + 服务。

04

面向新一代知识的数据基础设施

三个模块 · 一个闭环 · 架构演进而非另起炉灶

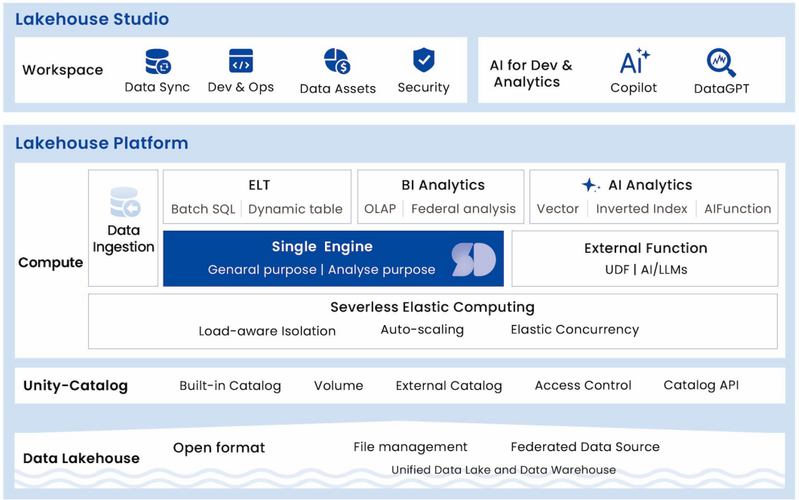


云器科技 AI Lakehouse：云中立的 Data + AI 平台

成立于 2021 年；以 Single-Engine 理念和通用增量计算为核心 —— 对标 Snowflake / Databricks，国内对标最完整的一家

定位：云中立的企业级数据平台 —— 一个引擎、一份数据，覆盖批 / 流 / 交互 / AI，对标海外 Snowflake 与 Databricks 的完整能力。不绑定任何云厂商，主流公有云与私有化同构部署；

基础架构：三层，一个引擎



核心产品优势

1	**开放湖仓作为底座**	兼顾结构化与非结构化数据：表、文件、向量、文档一个湖仓统一管理
2	**以 Apache Iceberg 为开放核心**	开放格式、统一 Catalog，一份数据多引擎互操作，不被锁定
3	**高性能引擎：开源的 10 倍**	C++ 向量化 Single-Engine，离线批处理比 Apache Spark 快 10 倍
4	**通用增量计算：统一流批交互**	只算变化的部分，一套语法、一个引擎、一份数据 —— 分钟级实时，接近离线的成本
5	**多云的企业级平台**	全托管 Shared-Everything；SaaS、专属云、私有化多种部署；Studio 一体化开发与治理

典型场景与核心客户价值

性能与成本　10 倍性能，TCO 大幅下降

汽车 / IoT 海量数据平台　千万级车端接入，PB 级秒级查询

大众 · 丰田 · 长安 · 长城

日志加工与分析　日均几十 TB，一个平台一份 SQL

数美科技

实时化与架构收敛　一套链路替代 Lambda，分钟级实时；多云同构

近实时加工　增量计算替代 Flink / Spark 双链路，成本降数倍

快手 · 高途 · 销售易

跨云迁移与多云统一　兼容阿里云技术栈，解除绑定；国内 / 海外一套

星星充电 · 畅捷通 · 螳螂科技

AI 数据与知识工程　从 Raw 到知识，支撑 Agent 落地

AI-ETL 与知识库　模型即 SQL 函数，语音 / CRM / 文档变成知识

物流企业 · 车企 CRM Agent

语义视图 + Agent 评估　文档蒸馏为语义视图；SingSight 可观测与评估

内容平台 · 本次案例一

V2 架构总览

知识平台是数据平台的演进，不是另起炉灶：管线、分层、增量、评估一脉相承；结构化数据本身就是知识的一部分

云器 V1

数据基座

- 统一数据基座（Lakehouse）
- 高性能引擎（10 倍）
- 通用增量计算（GIC）
- 解决性能、成本与实时性瓶颈
- 企业级平台，全托管

构建于

云器 V2 · AI 知识平台

1 AI-ETL

持续把原始数据加工成 AI-ready 数据：抽取、清洗、结构化与语义增强。

2 SingSight

AI 可观测与持续评估：Trace、数据集、实验与自动化评估闭环。

3 知识工程/蒸馏

把数据、文档和 Trace 蒸馏为结构化知识资产，供 Agent 使用、治理与审计。

4 AI 网关

统一、模型中立的大模型接入，具备权限、成本控制与审计

5 Agent 入口

Data Agent + Analytics Agent 作为主要入口

6 MCP / CLI 接口

标准接口，无缝接入企业已有的 Agent 生态

V2 架构：从数据平台到 AI 知识平台

包含 V1 全部能力，再加上 AI-ETL、Agent 可观测与评估，以及统一知识库

包含 V1 全部能力

开放湖仓、高性能引擎、增量计算、企业级 Studio

AI-ETL

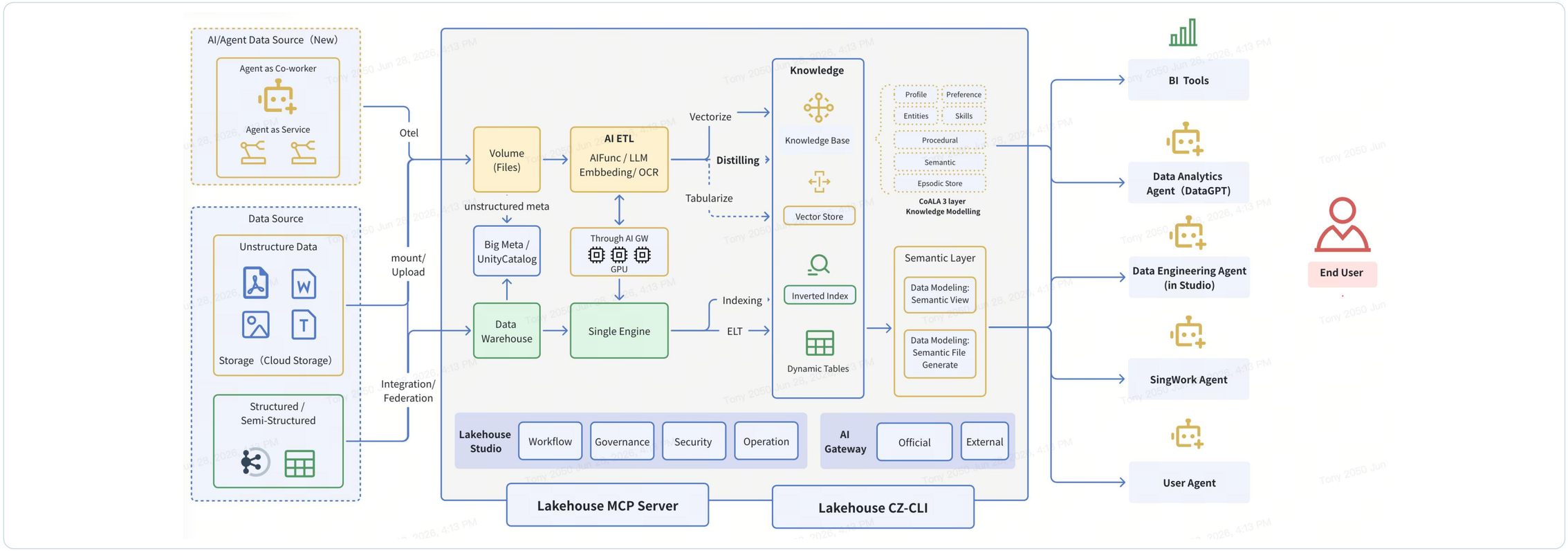
AI 函数调用通用或本地模型，在实时管道中处理异构数据

Agent trace与评估

基于 OTel 的 Agent 数据接入；全链路 LLM / 工具调用分析与评估

数据库 → 知识库

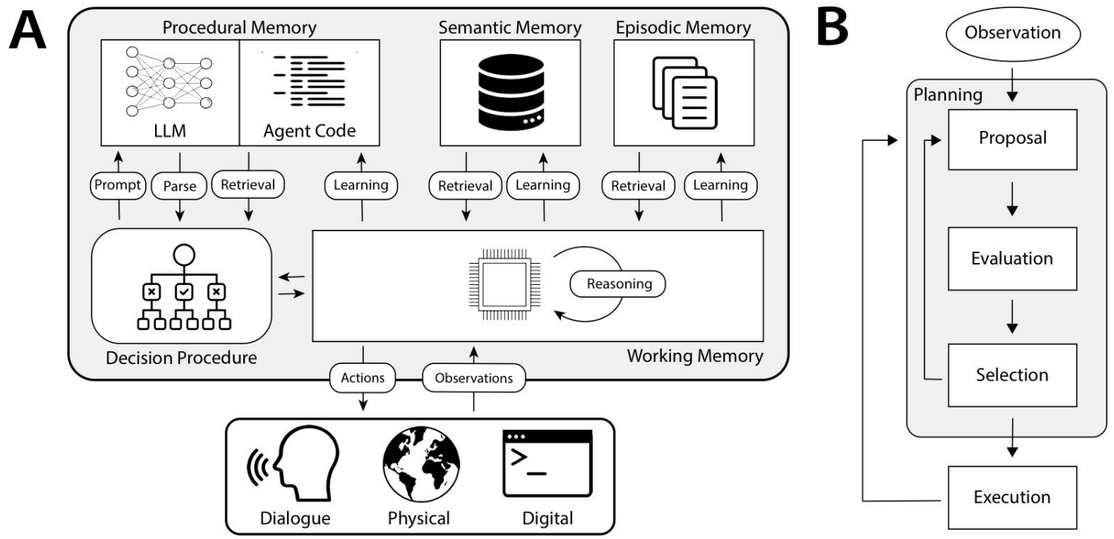
企业数据 + 清洗后的多模态数据 + 索引 + 向量，统一为知识库



知识的三种服务模式：语义视图 · 知识图谱 · 混合检索

CoALA（语言智能体认知架构）把知识视为 Agent 的显式上下文。我们把企业证据映射为语义含义与可复用的程序，再通过多种检索接口提供服务。

1 以 COALA 为参考架构



2 企业知识三层模型

程序性知识

可复用的流程、规则、技能与工具使用模式。企业如何行动。

语义知识

实体、概念、关系与业务上下文。企业知道什么。

情景（原始）证据

文档、对话、表、事件与多模态来源。让每个抽象都有证据可依。

3 多种知识服务模式

语义视图 面向结构化数据

兼容 Ossie 的语义视图为分析、BI 和 AI Agent 定义共享的数据集、指标、维度与关系。

知识图谱 面向共享的通用对象

领域实体、关系与血缘，为高价值知识建模，支持遍历与推理。

混合检索 面向通用知识

向量与标量索引支持在通用企业知识上的抽取、排序与上下文召回。

来源：Sumers 等，Cognitive Architectures for Language Agents，TMLR 2024，图 4（CC BY 4.0）；Apache Ossie（孵化中）规范。三层模型是对 CoALA 长期记忆的企业化改编。

三个模块，一个闭环：Eco-loop 的产品映射

为什么是这三个模块？因为 Trace（SingSight）、蒸馏（AI-ETL）、认证与服务（知识）正好构成 Eco-loop。



最佳切入场景

- 客服 / 销售 Agent
- 企业知识问答
- 运维 / 日志分析
- 产品 / 内容运营

THANKS

云器

演讲嘉宾：关涛

云器科技 联合创始人 & CTO

KEY TAKEAWAYS · 五个观点

1. 知识是数据领域的第三次革命，也是企业在 AI 三要素中最大的差异化来源。
2. 知识的表达（本体）目前没有统一标准，有五条不同的路线。
3. Raw Data 不等于 AI-ready Data：知识需要一个生产的过程。
4. 知识工程的本质，是一个从非结构化到结构化的过程。
5. 构建知识工程平台，依赖一套整体的基础设施。

扫码获取本次分享材料